

Solana integration, version 2

Thinking phase. Nothing in this document is built. It answers three questions and redraws the design with the transporter in the middle.

Three questions, short answers

Question	Short answer
1. Do we need an RPC node now?	No paid one. Program tests run in memory and devnet has a free public endpoint. Pick a provider before the mainnet pilot. Our code only needs one setting, so changing later is easy. (page 2)
2. Which QuickNode finance example fits best?	escrow , the Anchor v1 copy, as the base. Borrow fundraiser for pooled transport (target, deadline, refund) and managed-fund for fee rules (capped, fixed, no setter). (page 3)
3. How does the three-layer model work, and who pays?	Farmer, transporter and buyer each sign their own step. The buyer deposits once, and the program splits it by terms locked at commit. A small capped fee on completed orders pays for the platform. Nobody's money is lent or moved anywhere the rules do not allow. (pages 4 to 7)

What changed from version 1

- Version 1 said: extend the current program with a vault. Version 2 refines that: use the **escrow** example as the template for it, and upgrade Anchor from 0.30.1 to 1.2 as part of the same step.
- The transporter is now a full participant that signs pickup and delivery, not just a name in the database.
- Money design is explicit: one deposit, one locked three-way split, one capped fee.
- The roadmap has four stages. Pooled transport gets its own stage.

Who wins in this design

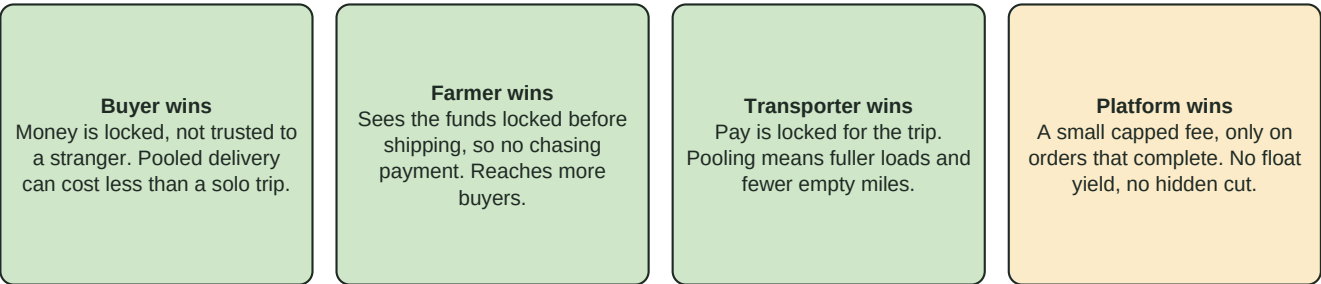
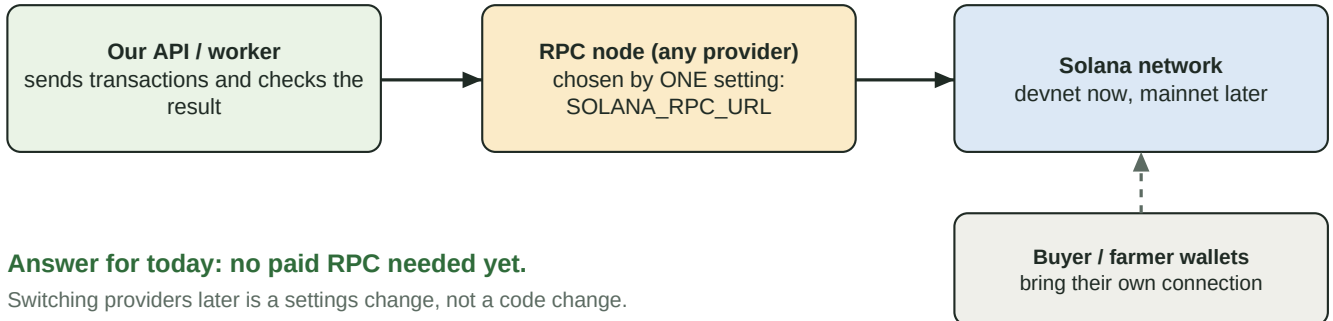


Figure 1. Everyone is paid from the same locked deposit; nobody is paid from someone else's pocket.

Do we need an RPC node now?

An RPC node is a server that answers questions about the blockchain and accepts transactions for it. Our payments code already talks to one through the single setting `SOLANA_RPC_URL`. Wallets (Phantom and similar) use their own connections, so you do not provide one for users.

An RPC node is the doorway between our servers and the Solana network



Answer for today: no paid RPC needed yet.

Switching providers later is a settings change, not a code change.

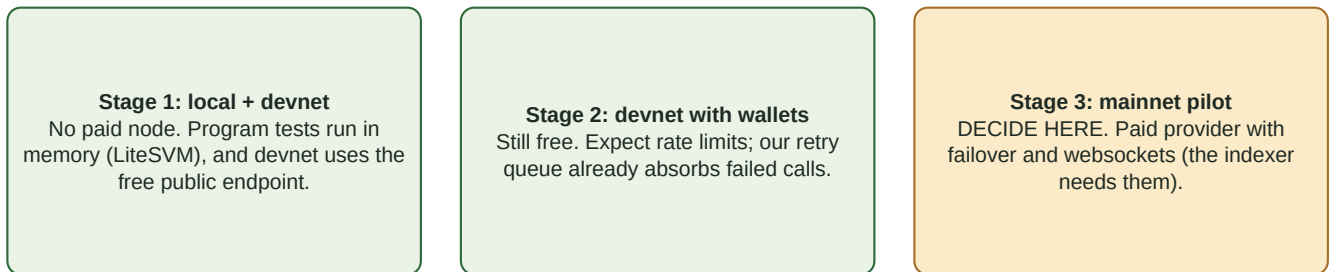


Figure 2. Where the RPC node sits, and what each stage needs.

When a paid provider becomes worth it

- **Real money:** the public endpoints are rate-limited and carry no promise of uptime, which is not acceptable once funds depend on it.
- **The indexer:** it needs a steady feed of events (websockets or frequent polling).
- **Failover:** a second endpoint to switch to if the first fails.
- **Real traffic:** many buyers acting at once.

What to compare: request limits and price, websocket support, failover options, devnet and mainnet in one plan, support. QuickNode is one candidate. I have not compared provider prices, so shortlist two or three at stage 3.

Which QuickNode example fits best?

The repo's finance folder has twelve examples. I read the code and README of escrow, fundraiser, managed-fund and token-swap; the others I judged from their folder names and descriptions.

Fit for our marketplace (my judgement, not a measurement)

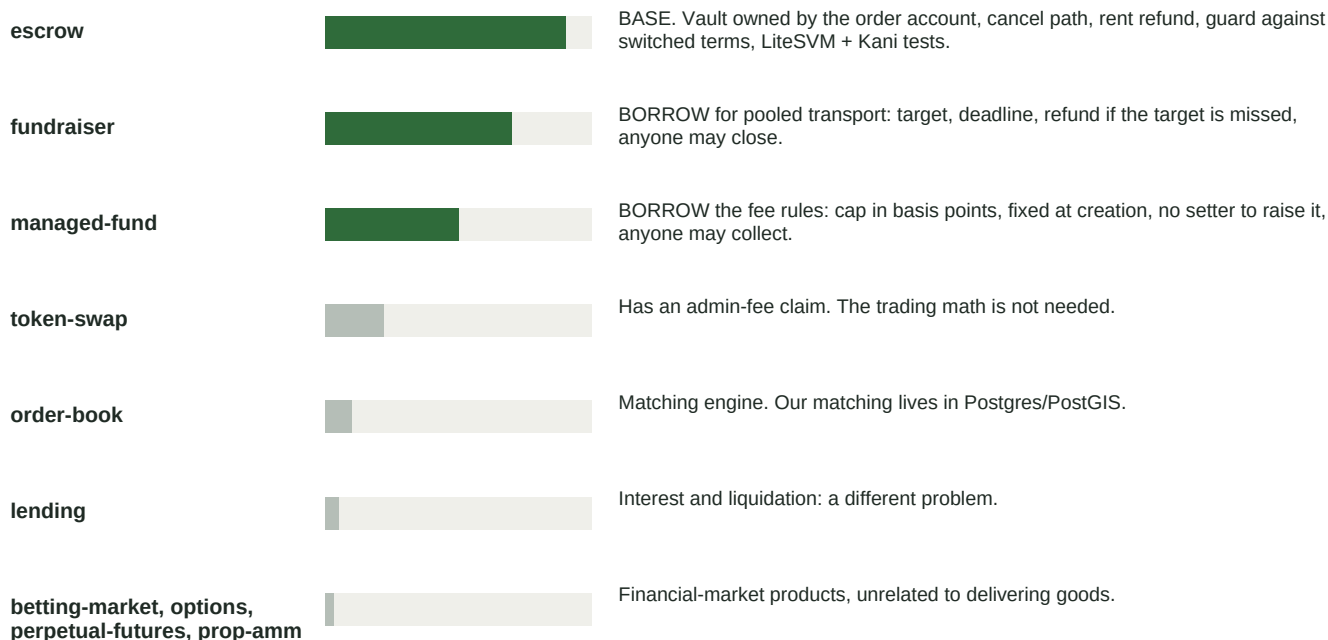


Figure 3. The score is a judgement of how much each example helps this project.

Verdict: build on escrow

Escrow is the smallest complete money program: one order account, one vault, three instructions, and tests. Its limit is that it swaps token for token in one atomic step, and our goods are physical, so there is a delivery in between. We keep its structure and add a delivery-confirmation step before release. It also charges no fee at all, so the fee split is ours to add, using managed-fund's rules.

Which copy of each example

Copy in the repo	What it is	For us
anchor-v1	Anchor 1.2.0, which its README calls the current stable release.	Use this. Same framework family as our code (ours is 0.30.1, so one upgrade step).
anchor (v2)	Anchor 2.0.0-rc.1: a release candidate.	Avoid for code that holds money until it is final.
native	No framework; everything by hand.	More code and more mistakes, no benefit for us.
quasar	A different framework.	Would mean learning a new toolchain. Not now.
kani-proofs	Model-checks the arithmetic for every input, not just test cases.	Copy the idea for our split: total in equals total out.

The three-layer model

Farmers and buyers are the two outer layers. The transporter is the middle layer: goods pass through them, and their signed pickup and delivery are the evidence the program waits for before releasing money.

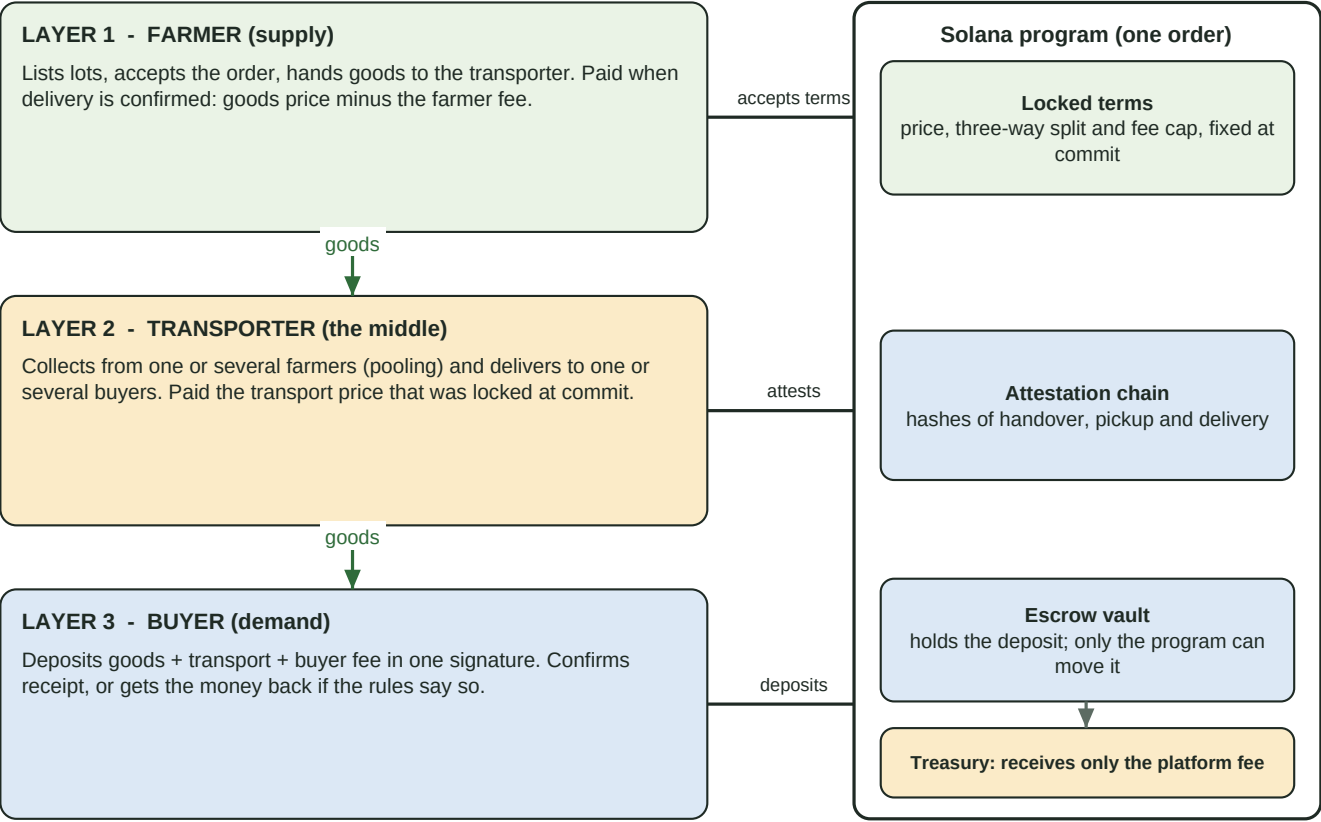


Figure 4. Green arrows are goods. Each layer signs only its own step; the program holds the money and the evidence.

One order, step by step

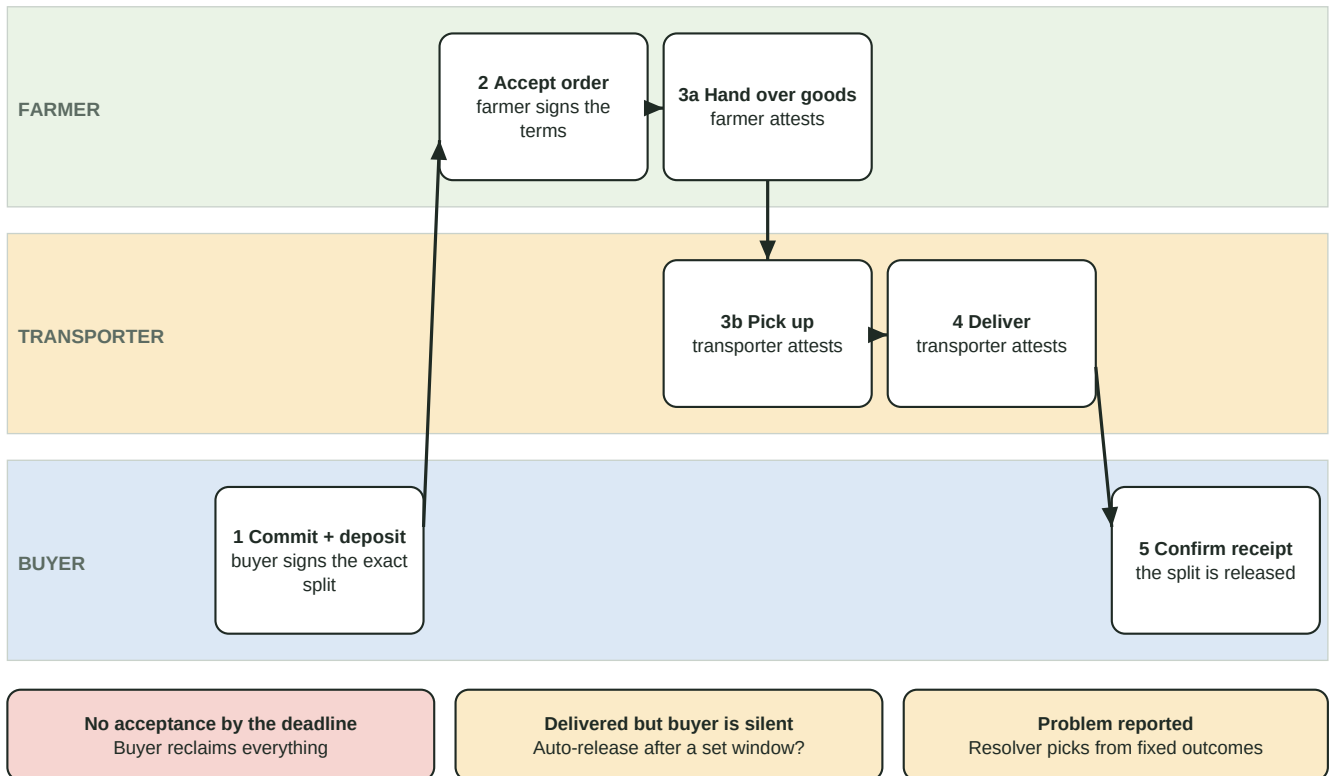


Figure 5. Who signs what. Boxes at the bottom are the unhappy paths.

What each unhappy path means

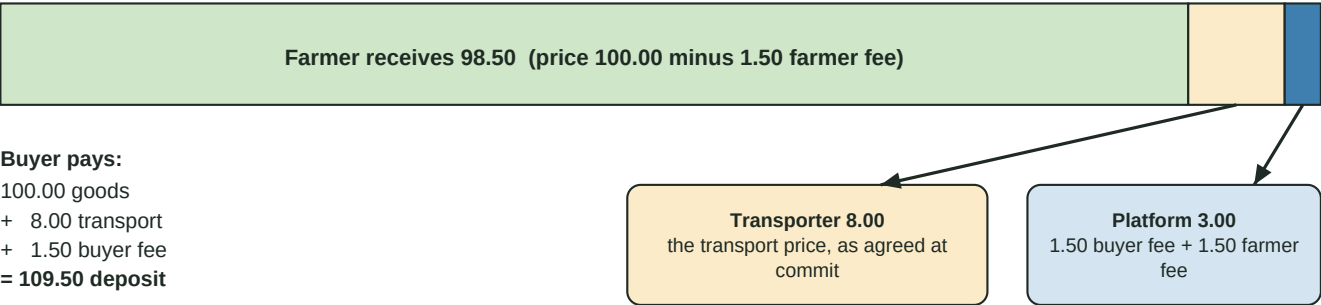
- **No acceptance:** the farmer never accepts. After the deadline the buyer reclaims the whole deposit, with no help from the platform.
- **Silent buyer:** delivery is attested but the buyer never confirms. Open question 3 on the last page: release automatically after a fixed window unless a problem is reported.
- **Problem reported:** a resolver can pick release, refund or the pre-agreed split. They cannot send money to a new address.

Instructions in this design

Instruction	Signed by	Template
make_order (deposit + locked split)	buyer	escrow: make_offer
accept_order	farmer	new
attest_handover / attest_pickup / attest_delivery	farmer / transporter / transporter	our record_attestation
confirm_receipt (releases the split)	buyer	escrow: take_offer
cancel_order (before acceptance)	buyer	escrow: cancel_offer
refund_after_deadline	buyer or anyone	fundraiser: refund
open_dispute / resolve_dispute	buyer or farmer / resolver	v1 plan
open_pool / join_pool / close_pool	transporter / order / anyone after deadline	fundraiser
collect_fees (pays the treasury only)	anyone	managed-fund

Who pays, and how the platform earns

Buyer signs ONE deposit of 109.50 (example numbers, not a proposal)



The program refuses the order unless the parts add up to the deposit, and the split can never change afterwards. Network fees (fractions of a cent) are sponsored by the platform out of its 3.00. Account rent is refunded when the order closes.

Figure 6. A worked example. The percentages are placeholders for a decision, not a recommendation.

Ways to earn, compared

Option	Who pays	Good	Risk	Verdict
A. Buyer service fee	Buyer	Simple and visible.	Raises the buyer's price.	Use, small.
B. Farmer commission	Farmer	Buyer's price stays flat.	Farmers may trade outside the platform if it is high.	Use, small.
C. Pooling success fee	Taken from realised savings	Platform earns only when pooling actually saves money.	Needs an honest price for the solo trip to measure against.	Stage 3.
D. Sponsor network fees	Platform, from A and B	Users never need to hold SOL.	Costs are tiny but real.	Use.
E. Dispute fee or bond	Losing party	Pays for the resolver.	Adds complexity.	Later.
F. Paid extras	Whoever buys them	Quality checks, analytics: off-chain, no custody.	Needs customers.	Later.
G. Earn yield on escrowed money	Nobody agreed	Looks like free money.	It uses people's funds; they could be locked or lost.	Never.

Recommended start: A plus B at a low combined rate, with option D paid out of it. Add C when pooling exists. Never F.

How it stays honest: no stealing

These are the rules the program should enforce, so honesty does not depend on a promise. None is built yet. Each one is copied from an example in the repo, or marked as new.

Rule	How the program enforces it	Copied from
1. The split is locked	Buyer signs exact amounts. The program checks the parts add up to the deposit with checked arithmetic, and no instruction can change them later.	escrow tests + Kani idea
2. The fee has a ceiling	A maximum (for example 5%) lives in the code. Each order stores its own fee, so a later change cannot touch open orders. No setter raises the cap.	managed-fund
3. Payout addresses are pinned	Farmer, transporter and treasury accounts are fixed at commit. The platform key can only press 'release' or 'refund', never choose a destination.	escrow account constraints
4. Terms cannot be switched	Accepting signs a hash of the terms, so an order cancelled and recreated with worse terms is refused.	escrow bait-and-switch guard
5. Deadline refund	If the order is not completed by its deadline, the buyer can reclaim without the platform's help.	fundraiser refund
6. Fees go only to the treasury	Anyone may trigger the fee collection, but the destination is fixed, so triggering it earns nothing.	managed-fund collect_fees
7. Resolver is boxed in	A dispute can only end as: release, refund, or the pre-agreed split among the same three parties. Never to a new address.	new design
8. The program cannot be swapped	Upgrade authority goes to a multisig, then time-locked or frozen. Without this, 'no stealing' is only a promise, because the owner could ship a new program.	plan, phase G

Rule 8 matters most in practice. Unless the program's upgrade key is controlled, a platform could replace the program with a different one, so a multisig and later a frozen or time-locked upgrade is part of going live.

Roadmap, version 2

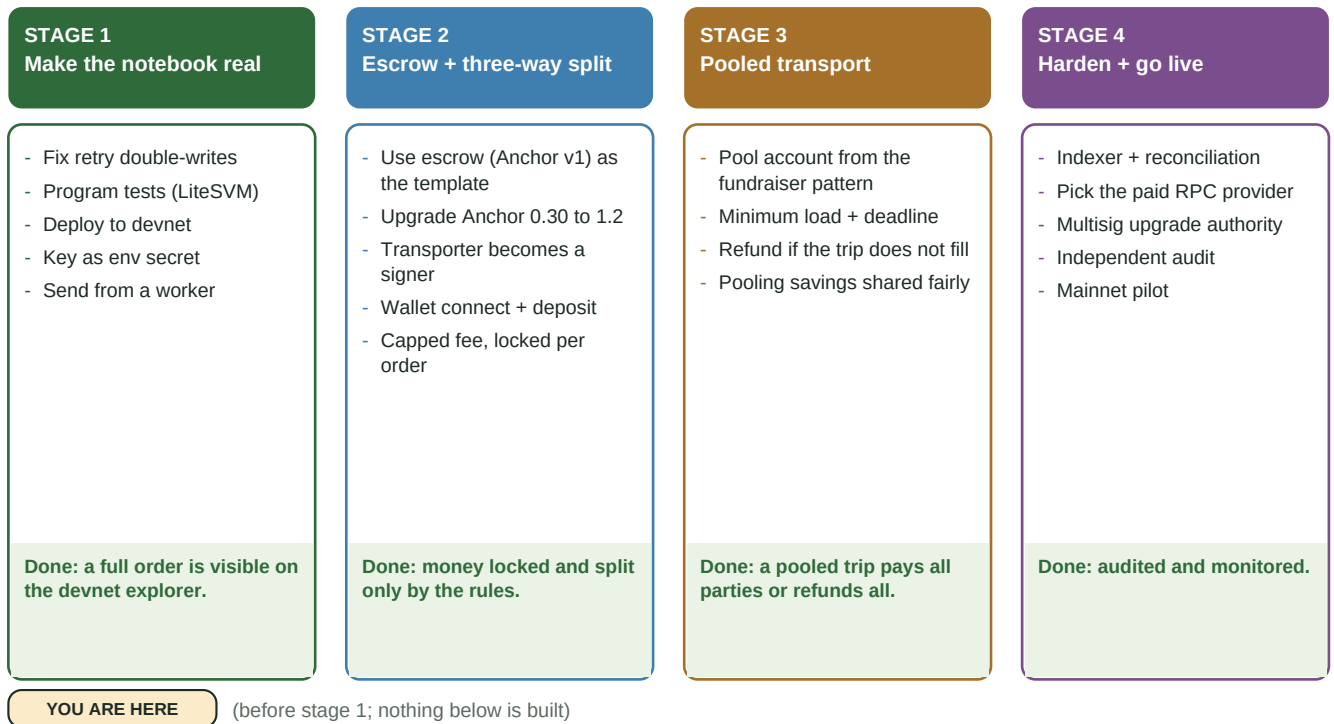


Figure 7. Four stages. Each one is useful alone, so you can stop after any of them.

Decisions I need from you

Question	Why it matters	My suggestion
1. Fee level and who pays it	Sets revenue and how competitive you are.	Start low, split between buyer and farmer, with a cap in the code. Adjust with real orders.
2. Who sets the transport price?	It is locked at commit, so someone must propose it.	The transporter quotes; the buyer sees it before signing. Algorithm suggestions later.
3. What if the buyer never confirms?	Money could sit forever.	Auto-release after a fixed window once delivery is attested and no problem is reported.
4. Who is the dispute resolver?	They decide contested orders.	Platform admin first, boxed to the fixed outcomes. Multisig panel later.
5. Token and on/off ramp	Same open questions as v1.	USDC on plain SPL Token, and choose a currency ramp before stage 2.
6. Is pooling in the first escrow release?	Pooling adds a second set of accounts and instructions to build and test.	No. Ship single-order escrow first (stage 2), pooling second (stage 3).

Next step if you agree: stage 1 is unchanged from version 1: fix the retry double-writes and add the program tests. It needs none of the decisions above.